

Temas Selectos de Física Computacional III:

Introducción a la ciencia de datos.

Clase 9: 1 Abril 2025

Dr. Leonid Serkin

1 Introducción

En esta clase, seguiremos estudiando los arboles de decisión y bosques aleatorios para el conjunto de datos MNIST, y generaremos nuestras propias imágenes de los dígitos.

2 Arboles y bosques para el conjunto MNIST

Ya conoces el conjunto MNIST. En esta ocasión usaremos todo el conjunto de datos MNIST usando la función `fetch_openml` de scikit-learn:

```
from sklearn.datasets import fetch_openml
dataset = fetch_openml("mnist_784")
X = dataset["data"]
y = dataset["target"]
```

Estudia el tamaño y estructura de los datos con `shape`, el tipos de datos con `type`, cuántos dígitos distintos hay con `np.unique`.

Asegúrate de convertir el target a enteros si aún es string

```
y = y.astype(int)
```

y dibuja algunos dígitos:

```
plt.imshow(X[i].reshape(28, 28), cmap='gray')
```

Ahora si vamos a entrenar los modelos, los pasos a seguir son:

1. Carga el conjunto de datos MNIST. De los 70,000 dígitos que tienes, selecciona solo los primeros 2000 dígitos.
2. Crea un conjunto de entrenamiento (1,500 dígitos) y un conjunto de prueba (500 dígitos).

3. Entrena un único árbol de decisión (con los parámetros predeterminados) en el conjunto de entrenamiento y luego calcula su precisión en el conjunto de prueba.
4. Aplica **gradient boosting** con los parámetros que usamos en la clase o usaste en tu tarea en el conjunto de entrenamiento y luego calcula su precisión en el conjunto de prueba.
5. Ahora entrena tu bosque aleatorio y calcula su precisión contra el conjunto de prueba.
6. Cómo se compara su precisión con los dos resultados previos?
7. Estudia cómo varía el rendimiento a medida que crece el número de estimadores. Prueba valores desde 10 hasta 100 (con incrementos de 10) para `n_estimators`.
8. Calcula la importancia de las 784 características de MNIST según tu bosque aleatorio (para calcular la importancia de cada árbol, puedes usar la importancia precomputada de `sklearn`, `tree.feature_importances_`).
9. A partir del ejercicio anterior, ahora deberías tener un arreglo con 784 importancias de características, una para cada una de las características en MNIST. Puedes remodelar este arreglo en una matriz de 28×28 valores (recuerda que las imágenes de MNIST son de 28×28 en blanco y negro). Puedes usar la biblioteca `seaborn` para visualizar un mapa de calor de esta matriz (es decir, una cuadrícula 2D donde los elementos tienen diferentes colores según su valor). El siguiente fragmento de código hace exactamente esto.

```
import seaborn as sns
feature_importances = get_feature_importances(clf)
sns.heatmap(np.reshape(feature_importances, (28, 28)), cmap='binary')
```

Ahora entrena un bosque aleatorio, extrae su importancia de características (`rf.feature_importances_`) y visualízalo. Te acuerdas del primer ejercicio de MNIST: qué características son más relevantes para distinguir los ceros de los unos? Y cuáles son las características más importantes ahora?

3 Usa tu propia imagen!

Una de las partes más emocionantes de esta clase es que puedes usar tu propio dígito escrito a mano, tomarle una foto y clasificarlo con el modelo que entrenaste!

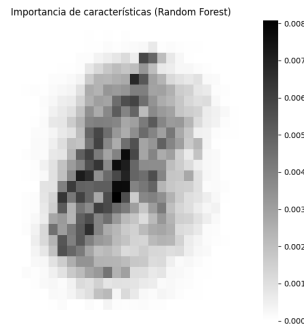


Figure 1: El resultado es que el clasificador está revisando los píxeles en la parte central de la imagen, ya que es donde normalmente habrá las principales diferencias.

A continuación, vamos a ver cómo preprocesar tu imagen para que tenga el mismo formato que las imágenes de MNIST.

Primero, usamos la biblioteca PIL para abrir la imagen, convertirla a escala de grises y redimensionarla a 28×28 píxeles. Luego, verificamos si el fondo es claro (blanco) y, si es así, lo invertimos para que coincida con el estilo del dataset (dígito blanco sobre fondo negro). Finalmente, aplanamos la imagen en un vector de tamaño 784:

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
def cargar_imagen(path):
    img = Image.open(path).convert("L")
    img = img.resize((28, 28))
    img_array = np.array(img)
    if img_array.mean() > 127:
        img_array = 255 - img_array
    img_flat = img_array.reshape(1, -1)
    return img_flat, img_array
```

Ahora simplemente carga tu imagen y visualiza para confirmar que todo se ha procesado correctamente:

```
plt.imshow(img_vis, cmap='gray')
```

Ahora tu dígito está en el mismo formato que los del conjunto MNIST, y está listo para ser clasificado por el modelo.

Una vez que procesamos nuestra imagen personalizada, es útil compararla visualmente con los dígitos reales del conjunto MNIST para ver si el estilo es parecido. Dibuja algunos ejemplos de tu dígito extraídos directamente de MNIST:

Ya habías entrenado un árbol de decisión con un subconjunto reducido de MNIST (solo 2000 imágenes), así que puedes usarlo y predecir el dígito de tu imagen directamente!

```
prediccion = clf.predict(X_digito)
```

Compara con un bosque aleatorio y visualiza las probabilidades que el modelo asigna a cada dígito:

```
probas = clf.predict_proba(X_personal)
for i, p in enumerate(probas[0]):
    print(f"Digito {i}: {p:.3f}")
```

Así obtienes una idea clara de cuán seguro está el modelo respecto a su predicción.

4 Ejercicios:

- 1.) Entrenar un `RandomForestClassifier` para cada dígito del 0 al 9 usando todos los 70,000 dígitos (train-test 60,000-10,000), obtener la importancia de características y visualizarla como un `heatmap`.
- 2.) Escribir a mano los dígitos del 0 al 9, tomarles foto y recortar cada uno en una imagen individual, preprocesar las imágenes (escala de grises, redimensionar, invertir si es necesario), clasificarlas con el modelo entrenado, mostrar la predicción para cada imagen, discutir sobre cuáles fueron clasificados correctamente y en cuáles se equivocó el modelo.